

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 3 SEPTEMBER 2019 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

SIGNATURE:

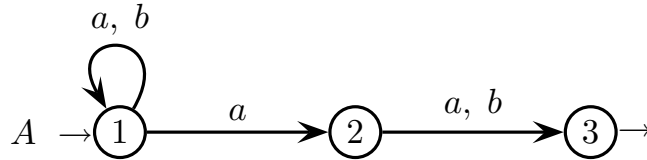
(or PERSON CODE)

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic automaton A below, over the two-letter alphabet $\Sigma = \{a, b\}$:



Answer the following questions:

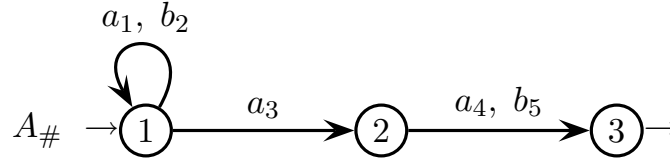
- (a) List all the strings of length 3 that are accepted by automaton A , and those (still of length 3) that are rejected by A .
 - (b) Use the Berry-Sethi (BS) method to derive a deterministic automaton A_d equivalent to automaton A . By using a systematic method, check whether automaton A_d is minimal, and if necessary minimize it.
 - (c) By using a systematic method, obtain an automaton $\overline{A}$ that accepts language $\overline{L(A)}$, i.e., the complement of language $L(A)$. List the accepting computations of $\overline{A}$ for the strings of length 3, which are rejected by automaton A .
 - (d) (optional) By using the node elimination (BMC) method, find a regular expression R that generates language $L(A)$. Based on R , provide an informal, i.e., written in English, concise and precise description of language $L(A)$.
-

Solution

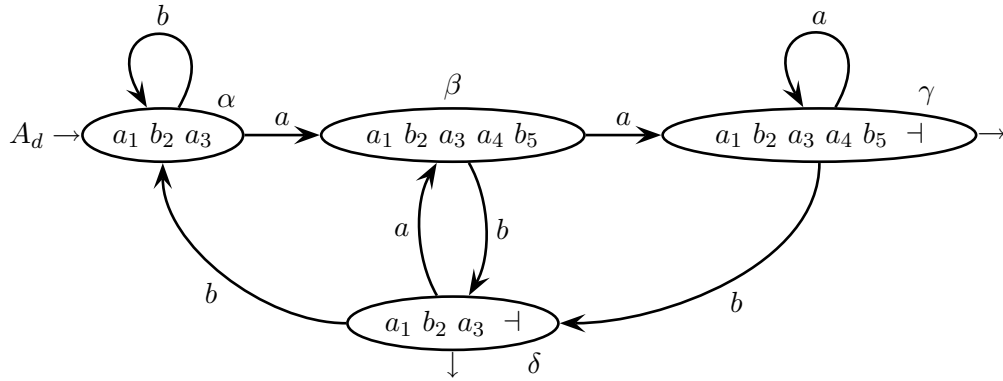
- (a) Over a 2-letter alphabet, the total number of strings of length 3 is $2^3 = 8$. Those accepted by automaton A are: aaa , aab , baa and bab (four in total). Those rejected by automaton A are: aba , abb , bba and bbb (four in total).

Automaton A visibly accepts all and only the strings of length ≥ 2 that have a letter a as their penultimate symbol. In fact, the four accepted strings above have a penultimate letter a and the four rejected ones have instead a letter b .

- (b) Here is the construction of the Berry-Sethi deterministic automaton A_d :



initials	a_1 b_2 a_3
generators	followers
a_1	a_1 b_2 a_3
b_2	a_1 b_2 a_3
a_3	a_4 b_5
a_4	$\dashv$
b_5	$\dashv$



Automaton A_d is deterministic by construction and has four states.

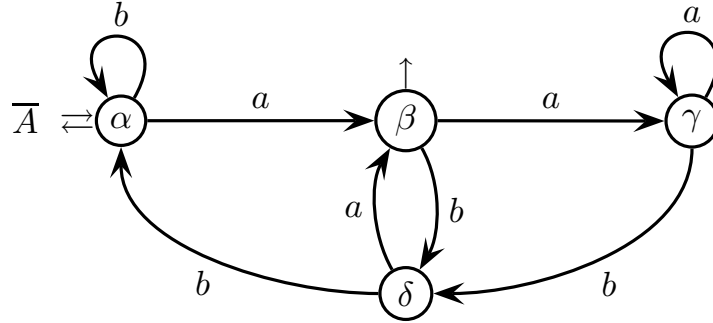
The deterministic automaton A_d is already minimal. See the the triangular undistinguishability table below (left):

β	(β, γ) (α, δ)		
γ			
δ			(γ, β) (δ, α)
	α	β	γ

The shaded cells are indexed by immediately distinguishable state pairs, thus not equivalent. Then solve the constrained cells, and since their contained state pairs (β, γ) and (α, δ) are immediately distinguishable, their indexing state pairs (α, β) and (γ, δ) are distinguishable, too, thus not equivalent either (right).

More speedily: the non-final states α and β can be distinguished by input a or b , and the final ones γ and δ can be distinguished by input a or b as well. Thus all the states are distinguishable from one another and the automaton is minimal.

- (c) An automaton $\overline{A}$ for the complement language is obtained from the deterministic automaton A_d (see the answer to question (b)), which happens to be already in the naturally complete form as each state has exactly a number of $|\Sigma| = 2$ outgoing transitions (so that it is not necessary to complete the automaton by adding an error state), by switching the final and non-final states of A_d :



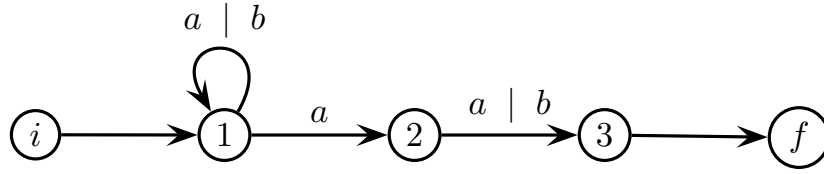
Notice that such a procedure may not be applied to automaton A , as this automaton is not deterministic. The complement automaton $\overline{A}$ is deterministic by construction, but it might not be minimal (check its minimality if you wish).

Here are the accepting computations of the complement automaton $\overline{A}$, for the four strings of length 3 that are rejected by automaton A :

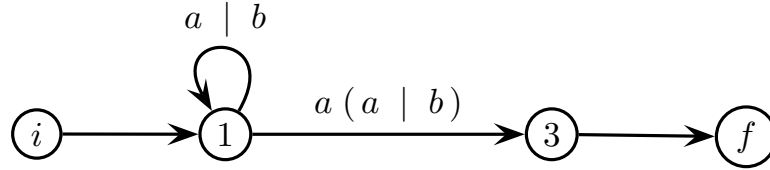
$$\begin{aligned}
 \alpha &\xrightarrow{a} \beta \xrightarrow{b} \delta \xrightarrow{a} \beta \\
 \alpha &\xrightarrow{a} \beta \xrightarrow{b} \delta \xrightarrow{b} \alpha \\
 \alpha &\xrightarrow{b} \alpha \xrightarrow{b} \alpha \xrightarrow{a} \beta \\
 \alpha &\xrightarrow{b} \alpha \xrightarrow{b} \alpha \xrightarrow{b} \alpha
 \end{aligned}$$

All such computations end in a final state (α or β) of $\overline{A}$, as expected.

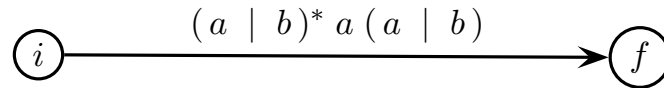
- (d) Here is a regular expression R , obtained from automaton A by node elimination (*BMC*). Unique initial and final nodes i and f , without ingoing and outgoing arcs, respectively:



Eliminate node 2:



Eliminate nodes 1 and 3:

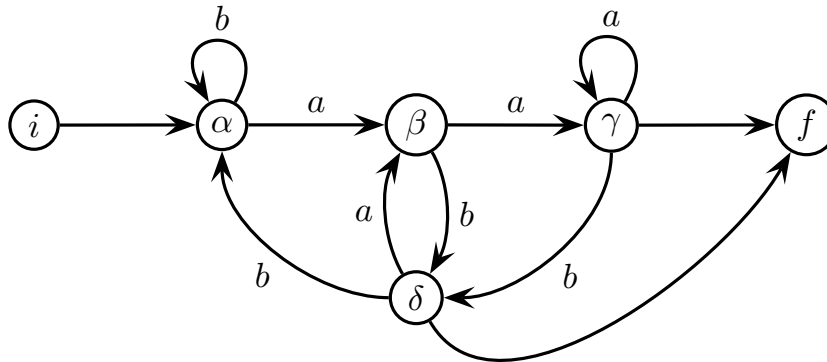


Therefore:

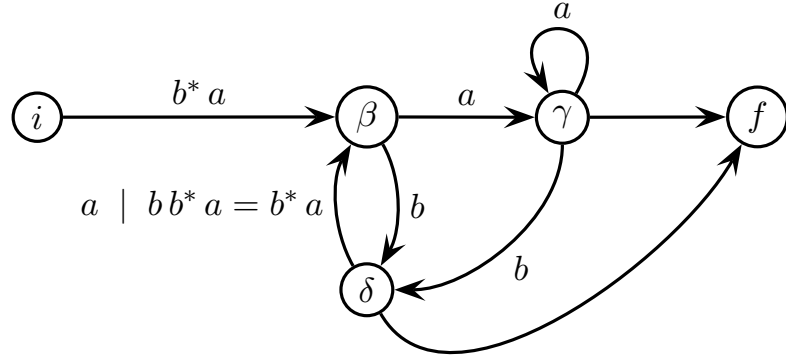
$$R = (a \mid b)^* a (a \mid b) = \Sigma^* a \Sigma$$

From the regular expression R it is evident that the strings in the language $L(A)$ contain at least two symbols, and that the penultimate symbol of such strings is bound to be a letter a (see also the answer to question (a)).

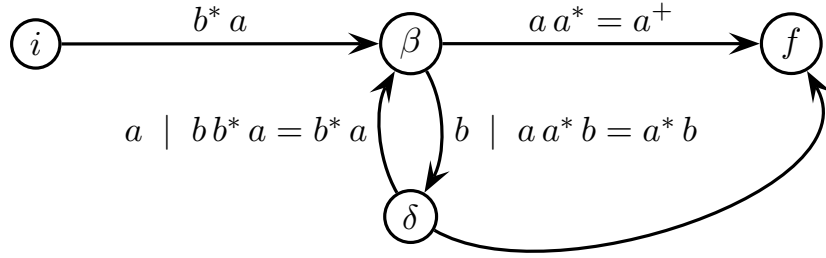
One could instead apply node elimination to the deterministic automaton A_d , though there are more states to eliminate. The process is somewhat longer:



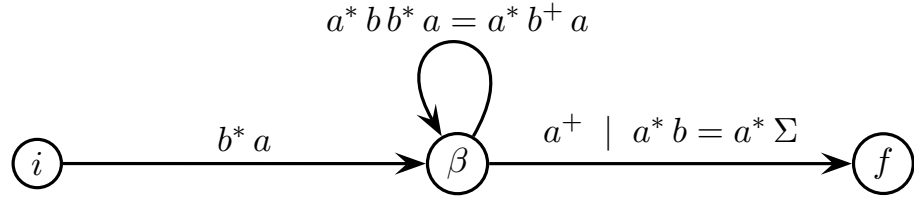
Eliminate node α :



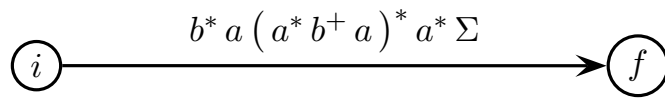
Eliminate node γ :



Eliminate node δ :



Eliminate node β :



Therefore, a regular expression R' equivalent to automaton A_d is the following:

$$R' = b^* a (a^* b^+ a)^* a^* \Sigma$$

Expression R' should be equivalent to expression R , i.e., $L(R') = L(R)$. For completeness, this can be verified through some algebraic manipulation of R' :

<i>(sub)expression before transforming</i>	<i>transformation</i>
$R' = b^* a (a^* b^+ a)^* a^* \Sigma$	expand star
$= b^* a (\varepsilon \mid a^* b^+ a \mid a^* b^+ a a^* b^+ a \mid \dots) a^* \Sigma$	distribute a left
$= b^* (a \mid a a^* b^+ a \mid a a^* b^+ a a^* b^+ a \mid \dots) a^* \Sigma$	replace $a a^*$ with a^+
$= b^* (a \mid a^+ b^+ a \mid a^+ b^+ a^+ b^+ a \mid \dots) a^* \Sigma$	factor a right
$= b^* (\varepsilon \mid a^+ b^+ \mid a^+ b^+ a^+ b^+ \mid \dots) a a^* \Sigma$	commute a and a^*
$= b^* (\varepsilon \mid a^+ b^+ \mid a^+ b^+ a^+ b^+ \mid \dots) a^* a \Sigma$	recompact star
$= \underbrace{b^* (a^+ b^+)^* a^*}_{\Sigma^*} a \Sigma$	replace $\dots$ with Σ^*
$= \Sigma^* a \Sigma = R$	

As for replacing the subexpression $b^* (a^+ b^+)^* a^*$ with the universal language Σ^* , note that the star $(a^+ b^+)^*$ generates the empty string ε or any string that begins and ends with (at least one) letter a and letter b , respectively, and then note that the prefix b^* and the suffix a^* allow the strings to optionally begin and end with (at least one) letter b and letter a , respectively. Therefore subexpression $b^* (a^+ b^+)^* a^*$ generates all the strings over alphabet Σ , i.e., the language Σ^* .

Of course, expression R' is definitely more complicated than expression R , thus it considerably obscures the simplicity of the language generated.

2 Free Grammars and Pushdown Automata 20%

1. Consider the ambiguous grammar G below, over a two-letter terminal alphabet $\{ a, b \}$ (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow a S b S \\ S \rightarrow a S \\ S \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Verify that grammar G is ambiguous, by finding a short string that has (at least) two syntax trees, and draw these two trees.
- (b) Consider the following grammar G' , similar to G but not identical (axiom S):

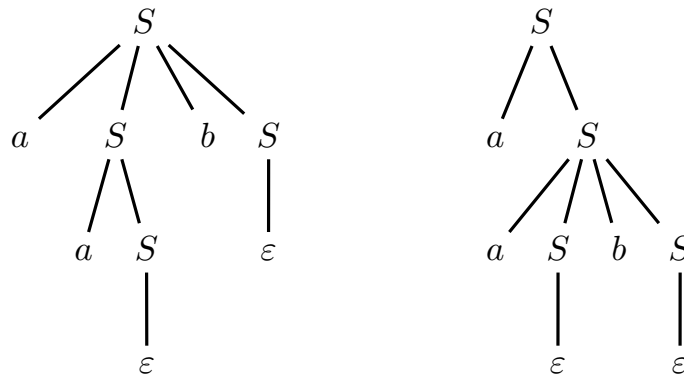
$$G' \left\{ \begin{array}{l} S \rightarrow S a S b \\ S \rightarrow S b \\ S \rightarrow \varepsilon \end{array} \right.$$

Say if grammar G' is ambiguous and reasonably justify your answer.

- (c) (optional) Is the intersection language $L(G) \cap L(G')$ regular? Is it context-free? Justify your answers.
-

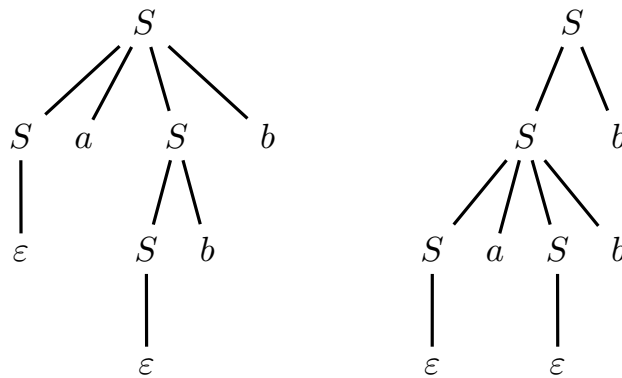
Solution

- (a) String $aa b$ is ambiguous for grammar G , with the two syntax trees below:



The string is the shortest ambiguous one and has only the two trees above. Longer strings may have more trees.

- (b) Grammar G' is also ambiguous. For instance, string abb has two syntax trees:



More generally: the first and third rule of G' generate the Dyck language with open bracket a and closed bracket b , while the second rule generates strings with more closed brackets b , which can match more than an open bracket a . For instance, with string abb (which we already know to be ambiguous):

$a b b$ $a b$ b

Such strings have more than one “deep structure” modeled by the grammar, i.e., more than one syntax tree, thus grammar G' is clearly ambiguous.

This reasoning also tells us that the ambiguous strings of G' are all and only those with exceedingly many closed brackets b . Furthermore, their ambiguity degree grows with the number of exceeding letters b , since there will be more and more matchings between the many letters b and the few letters a .

- (c) The intersection language is context-free, despite the context-free language family is not closed under intersection. In fact, the rules $S \rightarrow a S b S$ and $S \rightarrow \varepsilon$ of grammar G generate the Dyck language, with open bracket a and closed bracket b , while the rule $S \rightarrow a S$ of G adds more open brackets a without matching closed brackets b . Thus language $L(G)$ is the Dyck language with strings that may have exceedingly many open brackets a . Grammar G' is just an equivalent form of Dyck grammar, with concatenation on the left instead of on the right, and with exceedingly many closed brackets b instead of open ones a . Therefore, the intersection language $L(G) \cap L(G')$ is exactly the Dyck language (as there is not an excess of letters a over letters b or viceversa and thus all the brackets match exactly once), which is non-regular and is context-free.

2. A programming language fragment includes the expressions and variables that represent sets of binary integers, and the assignments of such expressions to the variables, according to these rules:

- A set expression can be one of the following:
 - a variable name, i.e., an identifier, modeled by the terminal symbol *id*, not to be further expanded
 - a list, possibly empty, of comma-separated binary integer constants; the list is enclosed within a pair of curly brackets ‘{ }’; a binary integer constant may not have any leading zeros
 - the union (keyword *union*) or the intersection (keyword *inters*) or the complement (keyword *compl*) of set (sub)expressions; union and intersection are binary operators and are *right*-associative, while complement is unary; by default the precedence between these set operators is, in decreasing order: complement, intersection and union
 - a pair of round brackets ‘()’ that enclose a set (sub)expression; as usual, by using round brackets one can change the default precedence defined between the set operators, see the fourth assignment in the sample sentence below
- An assignment statement, with the operator ‘:=’, assigns a set expression (on the right-hand side) to a variable (on the left-hand side).
- A sentence of the language is a non-empty list of semicolon-separated assignment statements.

Here is a sample sentence of the language (with four assignments):

```
s1 := { 101, 1001 };  
  
s2 := { 11 } union s1;  
  
s3 := compl s2 inters s1 union { 110 } union { 10 };  
  
s2 := compl ( s1 union compl { 100, 10 } )
```

Answer the following questions:

- (a) Write a grammar, of type *EBNF* and not ambiguous, that generates the above sketched language.
- (b) Test the correctness of your grammar by drawing the syntax subtree of the expression in the third assignment statement in the sample sentence above, i.e., the subtree of the expression:

```
compl s2 inters s1 union { 110 } union { 10 }
```

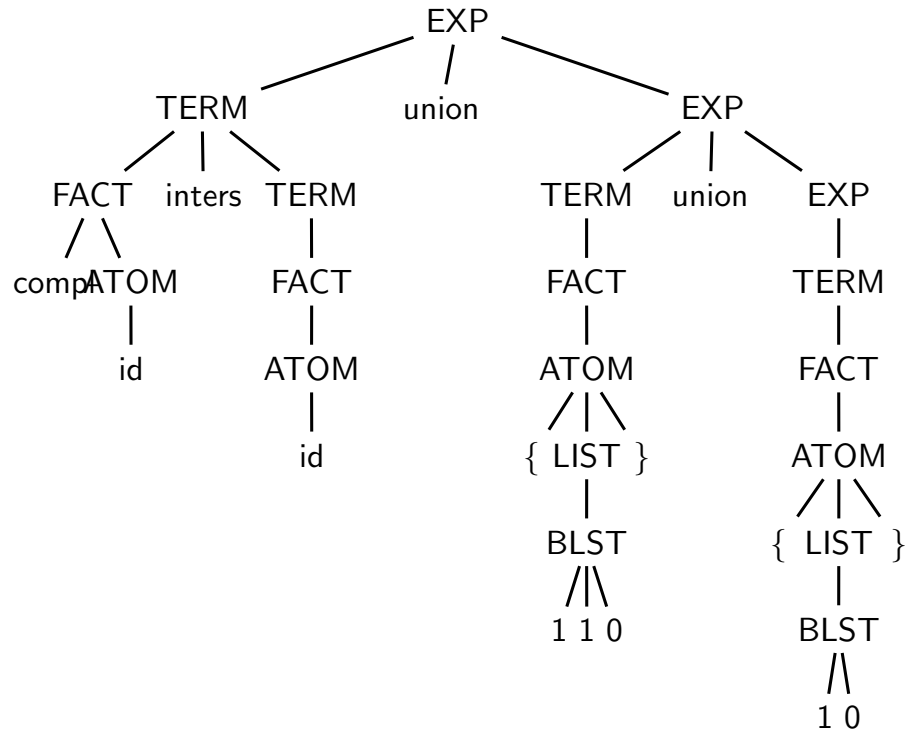
Solution

(a) Here is a viable grammar, of type *EBNF* and not ambiguous (axiom PRG):

$$\left\{ \begin{array}{l} \langle \text{PRG} \rangle \rightarrow \langle \text{STM} \rangle (';' \langle \text{STM} \rangle)^* \\ \langle \text{STM} \rangle \rightarrow \text{id} ':' \langle \text{EXP} \rangle \\ \langle \text{EXP} \rangle \rightarrow \langle \text{TERM} \rangle \text{ union } \langle \text{EXP} \rangle \mid \langle \text{TERM} \rangle \\ \langle \text{TERM} \rangle \rightarrow \langle \text{FACT} \rangle \text{ inters } \langle \text{TERM} \rangle \mid \langle \text{FACT} \rangle \\ \langle \text{FACT} \rangle \rightarrow \text{compl } \langle \text{ATOM} \rangle \mid \langle \text{ATOM} \rangle \\ \langle \text{ATOM} \rangle \rightarrow \text{id} \mid \{ ' [\langle \text{LIST} \rangle] ' \} \mid \{ (\langle \text{EXP} \rangle) \} \\ \langle \text{LIST} \rangle \rightarrow \langle \text{BLST} \rangle (',' \langle \text{BLST} \rangle)^* \\ \langle \text{BLST} \rangle \rightarrow '1' ('0' \mid '1')^* \mid '0' \end{array} \right.$$

The union and intersection rules are right-recursive, since both operators must be right-associative. The specified operator precedence is correctly modeled, see the subtree below. The grammar is somewhat standard for expressions.

(b) Here is the subtree of “compl s2 inters s1 union { 110 } union { 10 }” (root EXP):



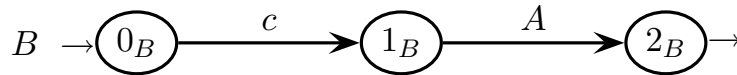
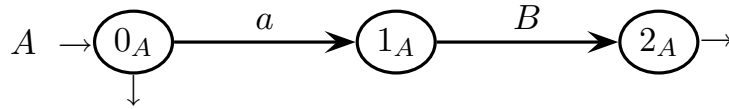
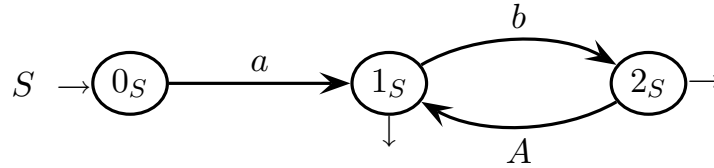
The EXP-TERM-FACT-ATOM rule chain models the operator precedence, which is union (lowest), intersection (middle) and complement (highest), namely:

$$\underbrace{\underbrace{\text{compl id} \text{ inters id}}_{\text{TERM}} \text{ union } \underbrace{\{ 110 \} \text{ union } \{ 10 \}}_{\text{EXP}}}_{\text{EXP}}$$

where variables s1 and s2 are schematized by id (notice union is right-associative).

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, over a three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and a three-letter nonterminal alphabet $V = \{ A, B, S \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) By drawing the complete pilot, determine whether grammar G is of type $ELR(1)$. Justify your answer.
- (b) By finding all the guide sets on the call arcs and exit arrows, determine whether grammar G is of type $ELL(1)$. Justify your answer.
- (c) By means of the Earley algorithm, analyze the valid string:

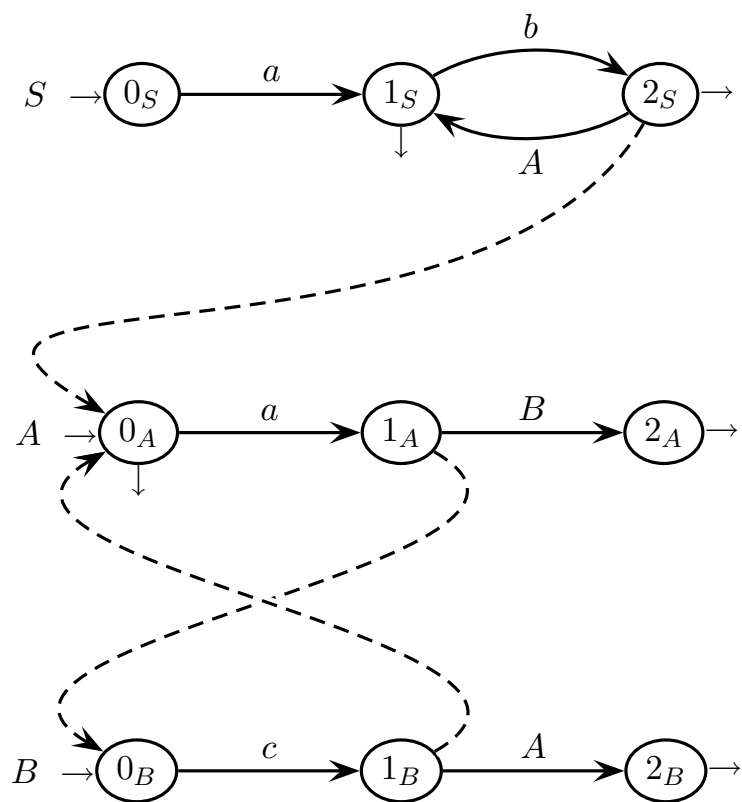
$$a b a c \in L(G)$$

and draw its syntax tree.

- (d) (optional) Is language $L(G)$ regular ? Justify your answer.

please draw here the pilot – question (a)

please write here the guide sets – question (b)



space for answering question (c)

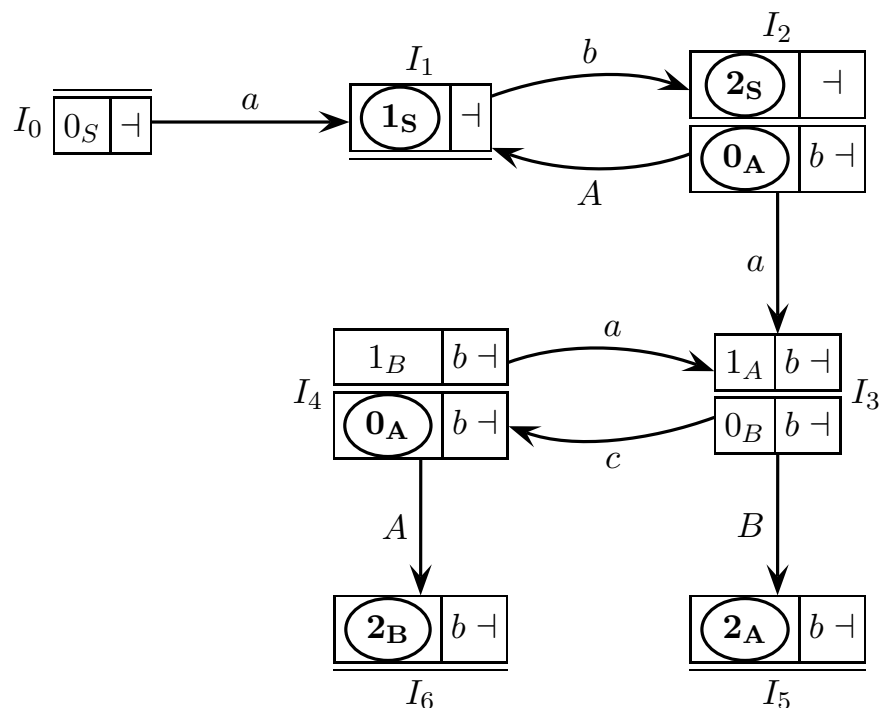
Earley vector of string $a b a c$ (to be filled in – the number of rows is not significant)

0*a*1*b*2*a*3*c*4

please draw here the tree of question (c) and answer question (d)

Solution

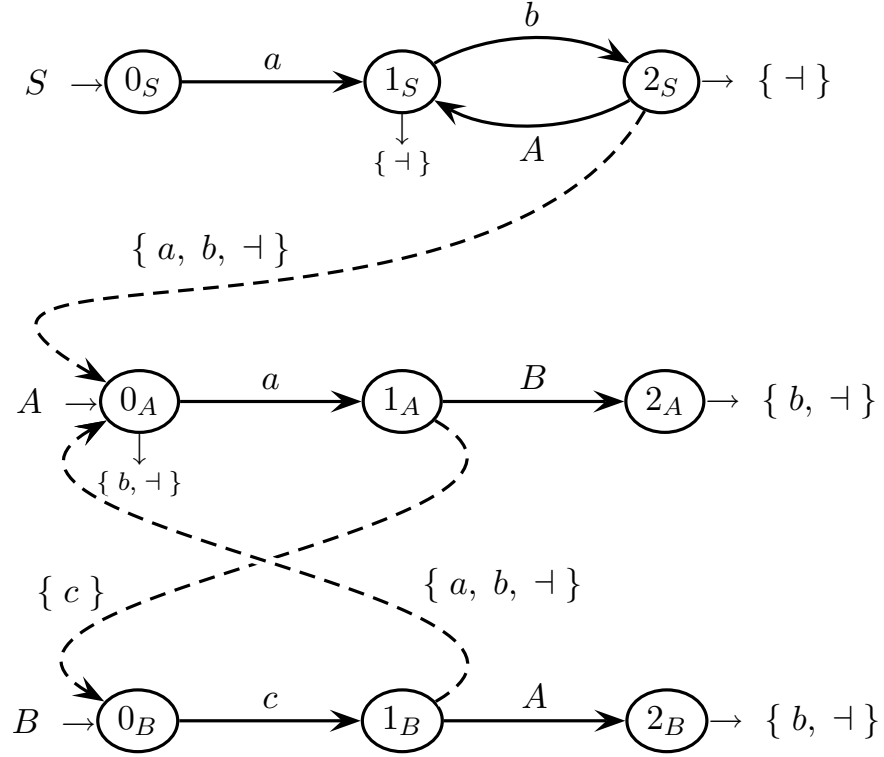
(a) Here is the complete pilot, with seven m-states:



In the m-state I_2 there is a reduce-reduce conflict on the terminator $\neg$, since the states 2_S and 0_A in I_2 are both final and share the look-ahead $\neg$. Therefore grammar G is not $ELR(1)$. There are not any other conflicts.

One can notice that grammar G is ambiguous. For instance, consider the valid string ab , which has two trees since nonterminal A is nullable. See also the answer to question (c). Ambiguity confirms that grammar G is not $ELR(1)$.

(b) Here is the complete *PCFG*:

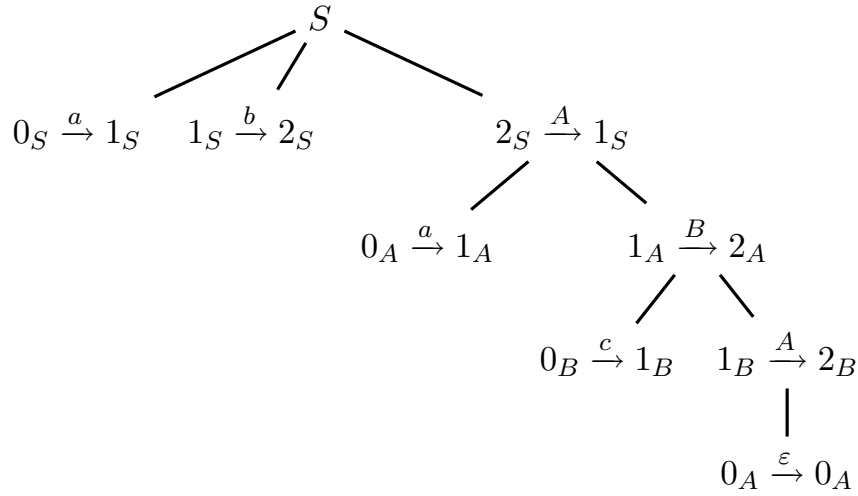


The only bifurcation states are 2_S and 0_A ; there is a guide set conflict on 2_S related to the terminator $\perp$ (state 0_A instead is conflict-free), thus the *PCFG* is not deterministic and grammar G is not *ELL*(1). After answering question (a), this was expected to happen, of course.

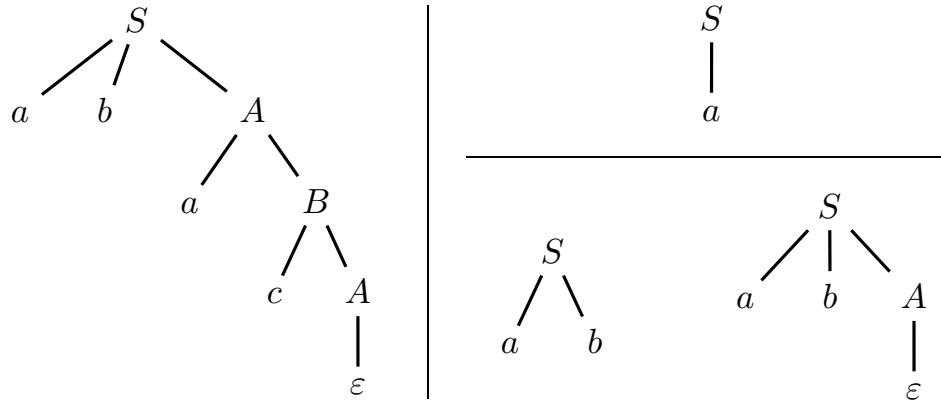
(c) Here is the complete Earley vector (the state cells unused are shaded):

0	a	1	b	2	a	3	c	4
0_S 0	1_S 0	2_S 0	0_A 2	1_S 0	1_A 2	0_B 3	1_B 3	0_A 4
							2_B 3	2_A 2
							1_S 0	

The final item $\langle 1_S, 0 \rangle$ in the Earley state $E(4)$ confirms that string $abac$ is valid. The syntax tree with machine paths is as follows:



and projecting it onto the grammar alphabets yields the usual syntax tree (left):



For completeness, the final item $\langle 1_S, 0 \rangle$ in the Earley state $E(1)$ proves that the prefix a is valid with one tree (top right). Furthermore, the two final items $\langle 2_S, 0 \rangle$ and $\langle 1_S, 0 \rangle$ in the Earley state $E(2)$ prove that the prefix ab is valid, too, in two different ways, i.e., with (at least) two syntax trees (bottom right); said differently, string ab is ambiguous. In fact, as said in question (a), grammar G is ambiguous and all the valid strings ab^+ are ambiguous, not just ab .

(d) Language $L(G)$ is regular. In fact, here are three *RE* for the machine languages:

$$\begin{aligned} L(S) &= a (b L(A))^* (b \mid \varepsilon) \\ L(A) &= a L(B) \mid \varepsilon \\ L(B) &= c L(A) \end{aligned}$$

which can be easily obtained by node elimination, for instance.

Therefore, by substitution:

$$L(A) = a c L(A) \mid \varepsilon$$

and by the Arden identity:

$$L(A) = (a c)^*$$

Thus, by substitution:

$$L(S) = a (b (a c)^*)^* (b \mid \varepsilon) = a (b (a c)^*)^* [b]$$

and in conclusion the language $L(G) = L(S)$ is regular. See also the tree of question (c), which is clearly linear.

This *RE* is slightly ambiguous (consider string $ab \in L(G)$), but of course there are also non-ambiguous equivalent forms. Here are a few:

$$a \left(\underbrace{b}_{\beta} \left(\underbrace{ac}_{\alpha} \right)^* \right)^* = a \left[\underbrace{b}_{\beta} \left(\underbrace{b}_{\beta} \mid \underbrace{ac}_{\alpha} \right)^* \right] = a \mid a b (b \mid a c)^*$$

The first one is obtained by removing the unnecessary optional b at the end of $L(S)$. The second one is obtained by the identity $(\beta \alpha^*)^* = [\beta (\beta \mid \alpha)^*]$. The third one just expands the optionality. None of them is ambiguous.

4 Language Translation and Semantic Analysis 20%

1. The source grammar G_s below (axiom S), over the two-symbol terminal alphabet $\Sigma = \{ '(', ')' \}$, generates as source language L_s the Dyck language over round parentheses $'(')$:

$$G_s \left\{ \begin{array}{l} S \rightarrow (S) T \\ S \rightarrow \varepsilon \\ T \rightarrow (S) S \\ T \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Write a syntax scheme G_τ (or a translation grammar) for the translation τ , over the source language L_s , defined as follows. In all the sequences of consecutive matching pairs of parentheses at any nesting level (including the top one):
 - the matching pairs that occur at odd positions, i.e., the first, third, fifth and so on, are left unchanged
 - the matching pairs that occur at even positions, i.e., the second, fourth, sixth and so on, are changed into square brackets

Two examples are reported below. For clarity, the bracket pairs of the first example are outlined, distinguishing the pairs that are left unchanged (underlined) from those that are changed into square brackets (overlined):

$$\tau \left(\left(\underbrace{() \overbrace{() ()}}_{\overbrace{() ()}} \right) () \right) = \left(() \overbrace{[() []]} \right) []$$

and

$$\tau \left((((()())())((())())((()()))) \right) = (((()[])())[([])()])[(()[])]$$

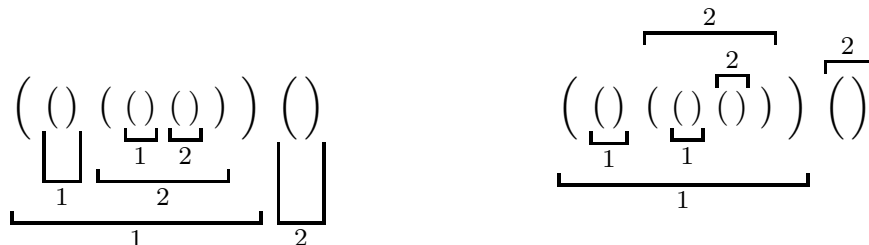
Do not change the source grammar G_s .

- (b) Draw the syntax translation tree of the valid source string of the first example above:

$$((()((()()))))()$$

Solution

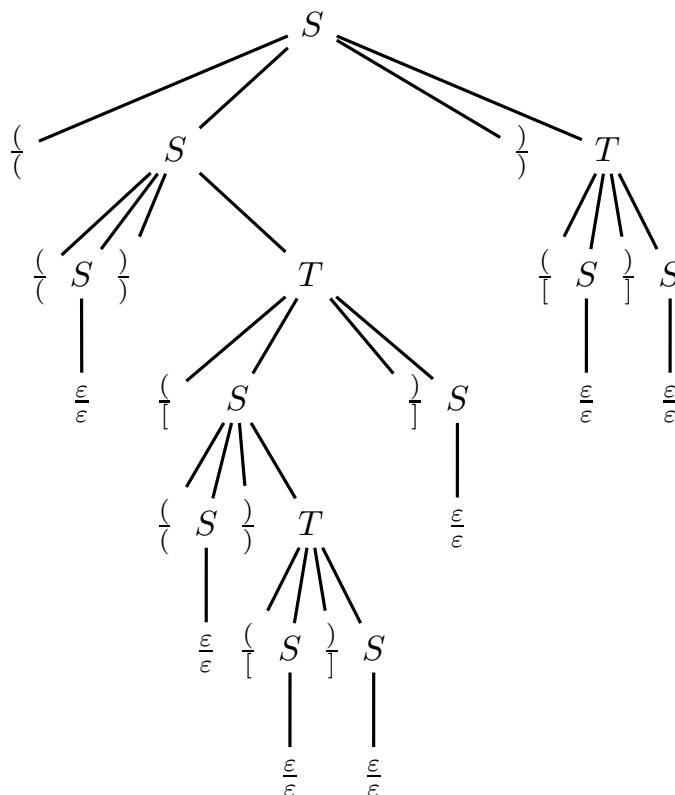
- (a) For better understanding, here is the position numbering of the bracket nests in the first example (on the left), and those in the even positions (here only position 2) flipped up (on the right), which have to be translated into square brackets:



Therefore, here is the target grammar G_t (axiom S), and the combined grammar G_τ aside:

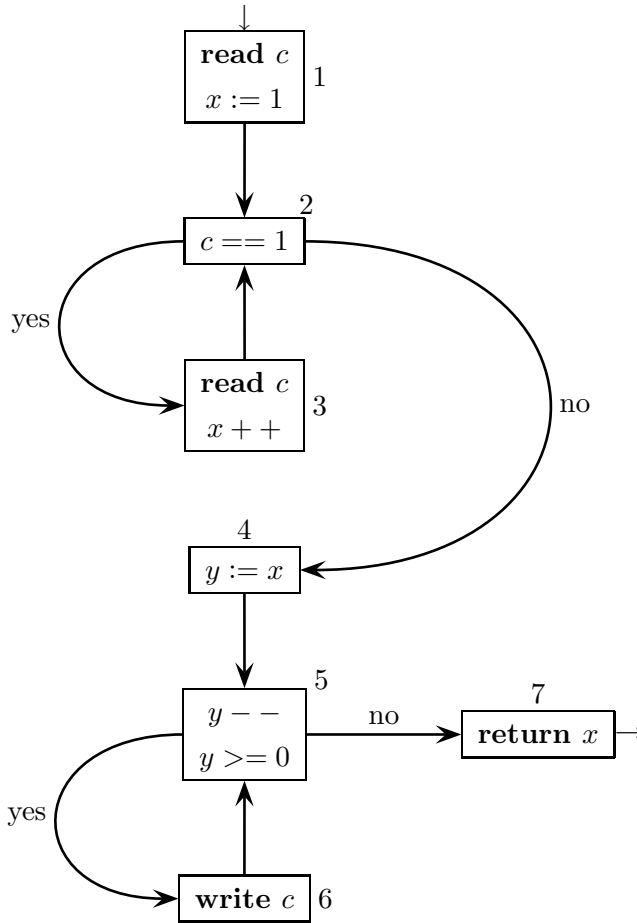
$$G_t \left\{ \begin{array}{l} S \rightarrow (S) T \\ S \rightarrow \varepsilon \\ T \rightarrow [S] S \\ T \rightarrow \varepsilon \end{array} \right. \quad \left| \quad G_\tau \left\{ \begin{array}{l} S \rightarrow \left(\frac{S}{\varepsilon} \right) T \\ S \rightarrow \frac{S}{\varepsilon} \\ T \rightarrow \left(\frac{S}{\varepsilon} \right) S \\ T \rightarrow \frac{S}{\varepsilon} \end{array} \right.$$

- (b) Here is the combined source-translation tree for the first example:



The reader may wish to draw the tree for the second example.

2. Consider the program Control Flow Graph (CFG) shown below, with seven nodes:



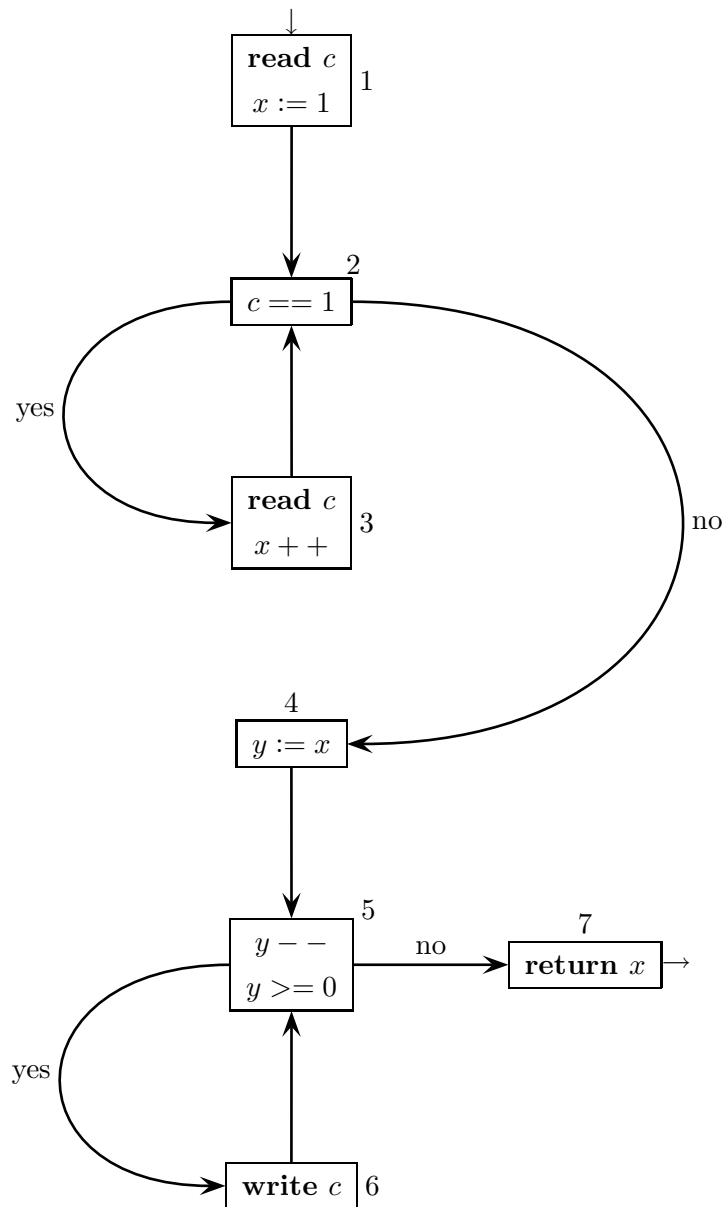
The program has three integer variables: x , y and c .

Answer the following questions (use the tables / drawings / spaces on the next pages):

- Disregarding the program semantics, write a regular expression R that represents the execution traces of the CFG over the node name alphabet $\{ 1, \dots, 7 \}$.
- Informally find the live variables at each node and write them by each node (the live variables of a node are those live at the node input). Can the found liveness intervals be exploited to optimize the memory allocation for the program ?
- Find the variables used and defined at each node. Then write the flow equations of the live variables, but only for the nodes 2, 5 and 6 (do not solve such equations). Verify that the solution found at point (b) satisfies the flow equations of nodes 2, 5 and 6. Remember that each node has two equations: *in* and *out*.
- (optional) Considering the *program semantics*, write an expression (not necessarily a regular expression), by refining the expression R of point (a), that defines more precisely the language of the execution traces of the CFG, over the alphabet $\{ 1, \dots, 7 \}$; also write a *grammar* G that generates such execution traces. If semantics is considered, do the liveness intervals of the variables change?

please write here the RE R – question (a)

please write here the live variables – question (b)



space for answering question (c)

tables of definitions and usages at the nodes

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	

(partial) system of data-flow equations

<i>node</i>	<i>in equations</i>	<i>out equations</i>
2		
5		
6		

space for the equation verification

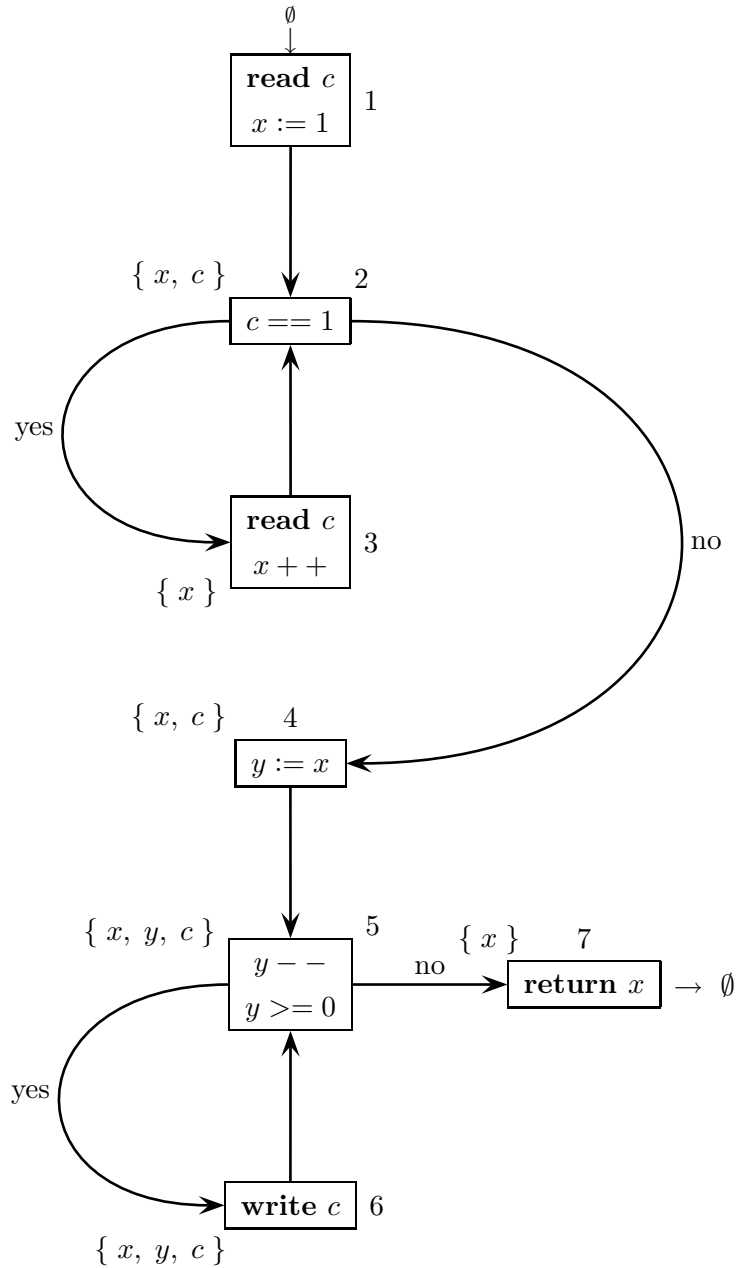
space for answering question (d)

Solution

(a) Here is the regular expression R , disregarding semantics:

$$R = 1\ 2\ (3\ 2)^*\ 4\ 5\ (6\ 5)^*\ 7$$

(b) Here are the live variables:



The liveness intervals of each variable pair are overlapping, thus no memory optimization is possible.

(c) Here are the tables of the definitions and usages at the nodes:

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1	$c \ x$	1	–
2	–	2	c
3	$c \ x$	3	x
4	y	4	x
5	y	5	y
6	–	6	c
7	–	7	x

and here is the (partial) system of data-flow equations:

<i>node</i>	<i>in equations</i>	<i>out equations</i>
2	$out(2) \cup \{c\}$	$in(3) \cup in(4)$
5	$(out(5) \setminus \{y\}) \cup \{y\}$	$in(6) \cup in(7)$
6	$out(6) \cup \{c\}$	$in(5)$

with the equation verification:

$$\begin{aligned}
in(2) &= out(2) \cup \{c\} = in(3) \cup in(4) \cup \{c\} \\
&= \{x\} \cup \{x, c\} \cup \{c\} = \{x, c\} && \text{as in point (b)} \\
in(5) &= out(5) \cup \{y\} = in(6) \cup in(7) \cup \{y\} \\
&= \{x, y, c\} \cup \{x\} \cup \{y\} = \{x, y, c\} && \text{as in point (b)} \\
in(6) &= out(6) \cup \{c\} = in(5) \cup \{c\} \\
&= \{x, y, c\} \cup \{c\} = \{x, y, c\} && \text{as in point (b)}
\end{aligned}$$

Notice that the variable y in the node 5 is orderly used and assigned (in $y++$), and then used again (in $y \geq 0$), therefore the term $use - def$ in the *in*-equation of node 5 evaluates to $\{y\}$ (not to $\emptyset$).

(d) Here is language L refined, considering semantics:

$$L_{ref} = 1 \ 2 \ (3 \ 2)^n \ 4 \ 5 \ (6 \ 5)^{n+1} \ 7$$

with $n \geq 0$. With this refinement, language L_{ref} is context-free (similar to $a^n b^n$) and is contained (strictly) in the language generated by the regular expression R of point (a). A grammar G for language L_{ref} is, for instance (axiom S):

$$G: S \rightarrow 1 \ 2 \ X \ 6 \ 5 \ 7 \quad X \rightarrow 3 \ 2 \ X \ 6 \ 5 \quad X \rightarrow 4 \ 5$$

However, the liveness intervals are the same as in question (b), thus no memory optimization can be done either after refining the language.